Loop Unrolling Example 1

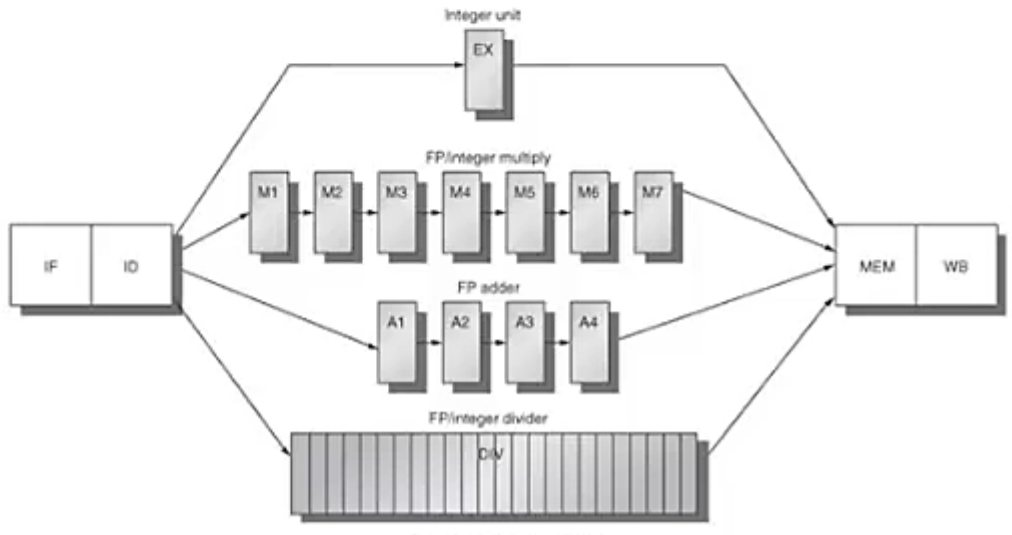
Loop Unroll Example 1

https://www.youtube.com/watch?v=3rydKkOWq7g

CS6810 -- Lecture 16. Lectures on Compiler-Based ILP

Assumptions

- Load: 2-cycles (1 cycle stall for consumer)
- FP ALU: 4-cycles (3 cycle stall for consumer; 2 cycle stall if the consumer is a store)
- One branch delay slot
- Int ALU: 1-cycle (no stall for consumer, 1 cycle stall if the consumer is a branch)



LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

Loop Example

```
for (i=1000; i>0; i--)
    x[i] = x[i] + s;
```

Source code

```
Loop:   L.D      F0, 0(R1)      ; F0 = array element
        ADD.D    F4, F0, F2     ; add scalar
        S.D      F4, 0(R1)      ; store result
        DADDUI   R1, R1,#-8     ; decrement address pointer
        BNE      R1, R2, Loop   ; branch if R1 != R2
        NOP
```

Assembly code

Convert C Code to Assembly Code

Loop Example

x[i] → F0

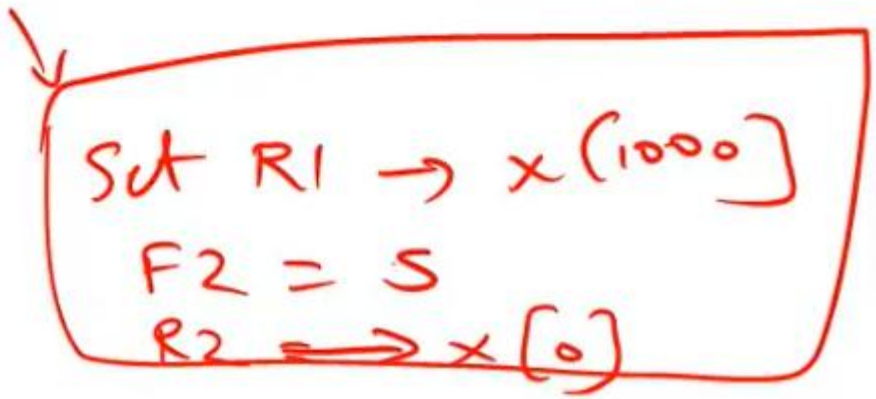
```
for (i=1000; i>0; i--)
    x[i] = x[i] + s;
```

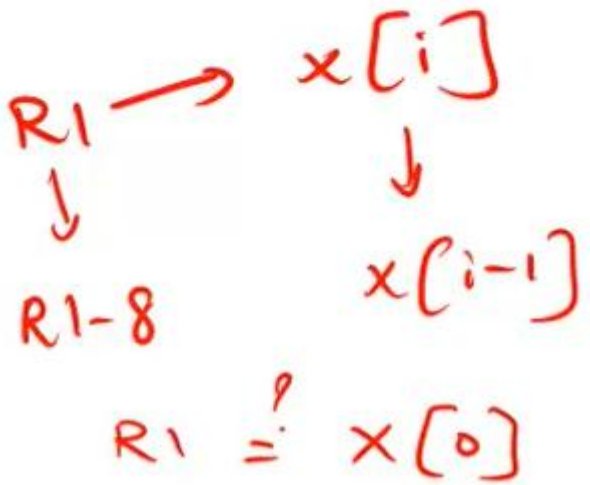
Source code

x[1000] → x[1]

```
Loop:  L.D      F0, 0(R1)      ; F0 = array element
       ADD.D    F4, F0, F2     ; add scalar
       S.D      F4, 0(R1)      ; store result
       DADDUI   R1, R1,#-8     ; decrement address pointer
       BNE      R1, R2, Loop   ; branch if R1 != R2
       NOP
```

Assembly code





6

Loop Example

LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

```
for (i=1000; i>0; i--)
    x[i] = x[i] + s;
```

Source code

```
Loop:  L.D      F0, 0(R1)       ; F0 = array element
       ADD.D    F4, F0, F2      ; add scalar
       S.D      F4, 0(R1)       ; store result
       DADDUI   R1, R1,#-8      ; decrement address pointer
       BNE      R1, R2, Loop    ; branch if R1 != R2
       NOP                      ; Just any instruction
```

Assembly code

```
Loop:  L.D      F0, 0(R1)       ; F0 = array element
       stall
       ADD.D    F4, F0, F2      ; add scalar
       stall
       stall
       S.D      F4, 0(R1)       ; store result
       DADDUI   R1, R1,#-8      ; decrement address pointer
       stall
       BNE      R1, R2, Loop    ; branch if R1 != R2
       stall
```

10-cycle schedule

1 stall

Loop Example

LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

```
for (i=1000; i>0; i--)
    x[i] = x[i] + s;
```

C Source code

```
Loop:   L.D      F0, 0(R1)      ; F0 = array element
        ADD.D    F4, F0, F2     ; add scalar
        S.D      F4, 0(R1)      ; store result
        DADDUI   R1, R1,#-8     ; decrement address pointer
        BNE      R1, R2, Loop   ; branch if R1 != R2
        NOP
```

Assembly code

3 lines of work
LD-ADD-SD

5 assembly instr

10-cycle schedule

10-cycles

```
Loop:   L.D      F0, 0(R1)      ; F0 = array element
        stall
        ADD.D    F4, F0, F2     ; add scalar
        stall
        stall
        S.D      F4, 0(R1)      ; store result
        DADDUI   R1, R1,#-8     ; decrement address pointer
        stall
        BNE      R1, R2, Loop   ; branch if R1 != R2
        stall
```

10 cycles

Smart Schedule

LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

```
Loop:  L.D      F0, 0(R1)
       stall
       ADD.D    F4, F0, F2
       stall
       stall
       S.D      F4, 0(R1)
       DADDUI   R1, R1,#-8
       stall
       BNE      R1, R2, Loop
       stall
```



```
Loop:  L.D      F0, 0(R1)
       DADDUI   R1, R1,#-8
       ADD.D    F4, F0, F2
       stall
       BNE      R1, R2, Loop
       S.D      F4, 8(R1)
```

- By re-ordering instructions, it takes 6 cycles per iteration instead of 10
- We were able to violate an anti-dependence easily because an immediate was involved
- Loop overhead (instrs that do book-keeping for the loop): 2
  Actual work (the ld, add.d, and s.d): 3 instrs
  Can we somehow get execution time to be 3 cycles per iteration?

8

Smart Schedule

LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

3 cyc/iter

```
Loop:  L.D      F0, 0(R1)
       stall
       ADD.D    F4, F0, F2
       stall
       stall
       S.D      F4, 0(R1)
       DADDUI   R1, R1,#-8
       stall
       BNE      R1, R2, Loop
       stall
```

```
Loop:  L.D      F0, 0(R1)
       DADDUI   R1, R1,#-8
       ADD.D    F4, F0, F2
       stall
       BNE      R1, R2, Loop
       S.D      F4, 8(R1)
```

3 instr work → 6 cycles

- By re-ordering instructions, it takes 6 cycles per iteration instead of 10
- We were able to violate an anti-dependence easily because an immediate was involved
- Loop overhead (instrs that do book-keeping for the loop): 2

Actual work (the ld, add.d, and s.d): 3 instrs
Can we somehow get execution time to be 3 cycles per iteration?

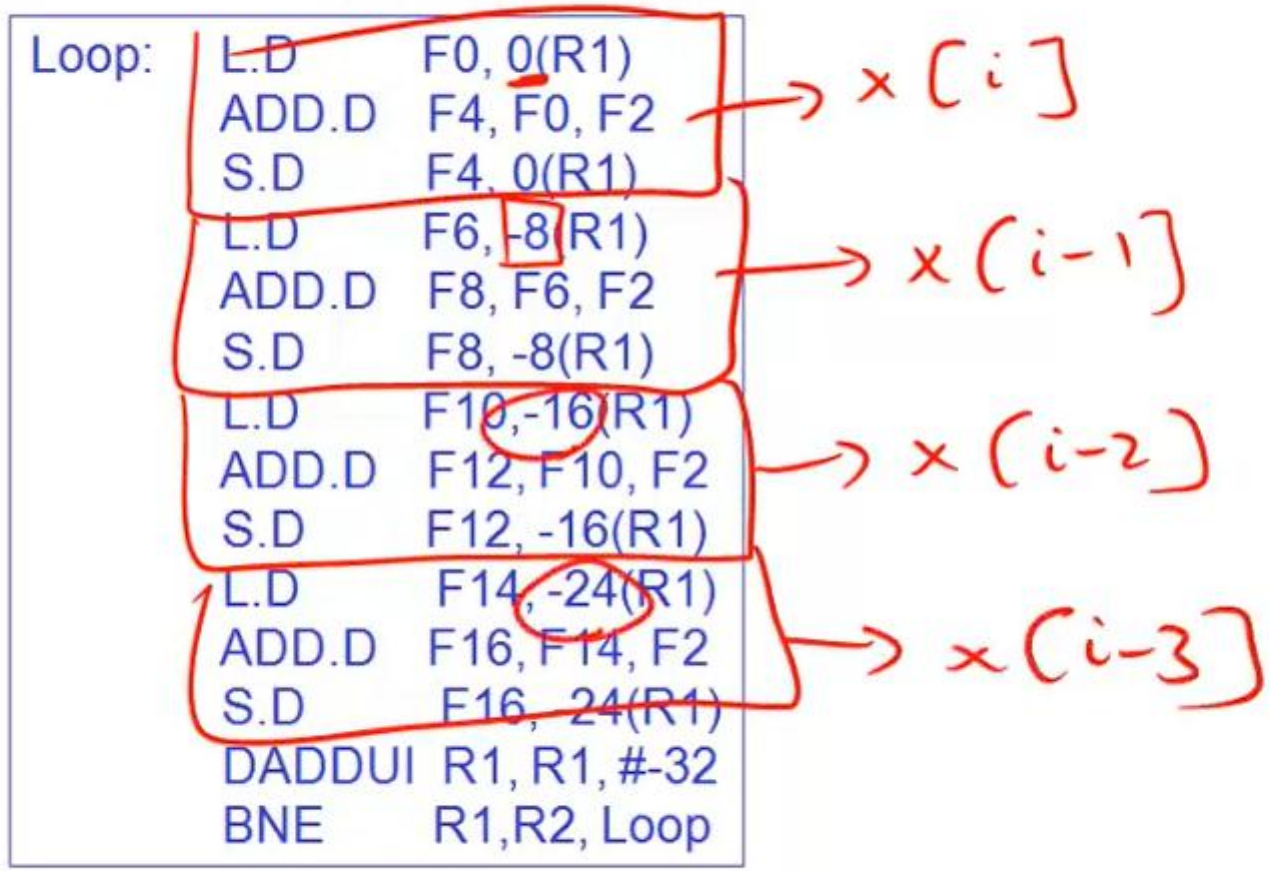
Loop Unrolling

```
Loop:   L.D      F0, 0(R1)
        ADD.D    F4, F0, F2
        S.D      F4, 0(R1)
        L.D      F6, -8(R1)
        ADD.D    F8, F6, F2
        S.D      F8, -8(R1)
        L.D      F10,-16(R1)
        ADD.D    F12, F10, F2
        S.D      F12, -16(R1)
        L.D      F14, -24(R1)
        ADD.D    F16, F14, F2
        S.D      F16, -24(R1)
        DADDUI   R1, R1, #-32
        BNE      R1,R2, Loop
```

Check once when completing unroll, use -32 value

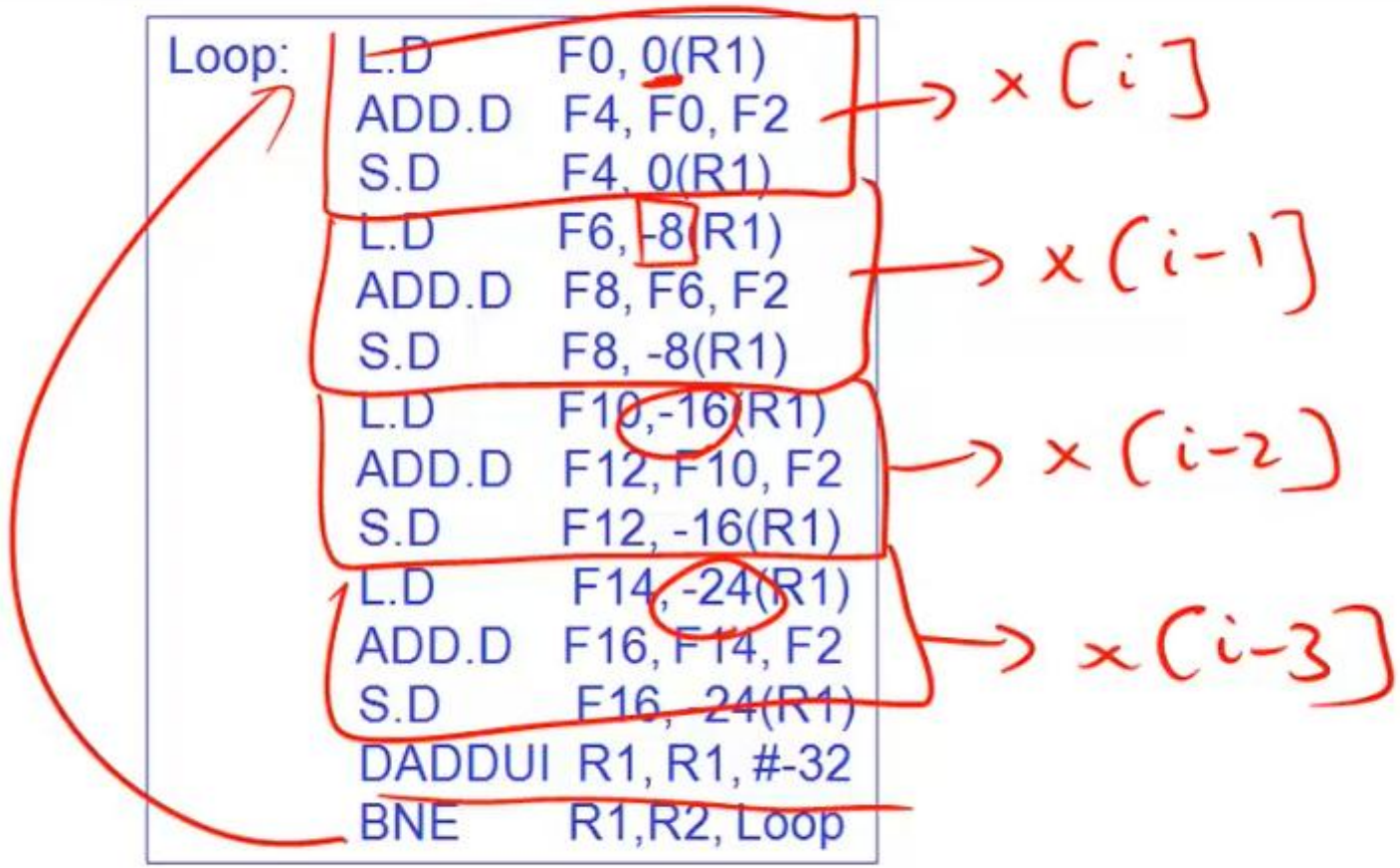
- Loop overhead: 2 instrs; Work: 12 instrs
- How long will the above schedule take to complete?

Loop Unrolling



- Loop overhead: 2 instrs; Work: 12 instrs
- How long will the above schedule take to complete?

Loop Unrolling



- Loop overhead: 2 instrs; Work: 12 instrs
- How long will the above schedule take to complete?

Scheduled and Unrolled Loop

Instruction Reordering To Prevent Data Hazard

```
Loop:  L.D      F0, 0(R1)
       L.D      F6, -8(R1)
       L.D      F10,-16(R1)
       L.D      F14, -24(R1)
       ADD.D    F4, F0, F2
       ADD.D    F8, F6, F2
       ADD.D    F12, F10, F2
       ADD.D    F16, F14, F2
       S.D      F4, 0(R1)
       S.D      F8, -8(R1)
       DADDUI   R1, R1, # -32
       S.D      F12, 16(R1)
       BNE      R1,R2, Loop
       S.D      F16, 8(R1)
```

LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

BNE requires 1 STALL to Complete
Last Stall is replaced by S.D

- Execution time: 14 cycles or 3.5 cycles per original iteration

Scheduled and Unrolled Loop

```
Loop:   L.D      F0, 0(R1)
        L.D      F6, -8(R1)
        L.D      F10,-16(R1)
        L.D      F14, -24(R1)
        ADD.D    F4, F0, F2
        ADD.D    F8, F6, F2
        ADD.D    F12, F10, F2
        ADD.D    F16, F14, F2
        S.D      F4, 0(R1)
        S.D      F8, -8(R1)
        DADDUI   R1, R1, # -32
        S.D      F12, 16(R1)
        BNE      R1,R2, Loop
        S.D      F16, 8(R1)
```

no dep

LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

- Execution time: 14 cycles or 3.5 cycles per original iteration

Scheduled and Unrolled Loop

```
Loop:  L.D      F0, 0(R1)
       L.D      F6, -8(R1)
       L.D      F10,-16(R1)
       L.D      F14, -24(R1)
       ADD.D    F4, F0, F2
       ADD.D    F8, F6, F2
       ADD.D    F12, F10, F2
       ADD.D    F16, F14, F2
       S.D      F4, 0(R1)
       S.D      F8, -8(R1)
       DADDUI   R1, R1, # -32
       S.D      F12, 16(R1)
       BNE      R1,R2, Loop
       S.D      F16, 8(R1)
```



LD -> any : 1 stall
FPALU -> any: 3 stalls
FPALU -> ST : 2 stalls
IntALU -> BR : 1 stall

14 cyc
4 iter worth of work
3.5 cyc/iter
3 cyc/iter

- Execution time: 14 cycles or 3.5 cycles per original iteration